

GameService

startNewGame(players: List<Player>, amountRounds: Int) : Unit

Allgemeine Beschreibung:

Die Methode startNewGame initialisiert ein neues Spiel „Schiebe-Poker“. Dabei werden die teilnehmenden Spieler registriert, das Kartendeck erstellt und gemischt, die Karten verteilt sowie die gewünschte Rundenanzahl festgelegt.

Vorbedingungen:

- Es darf kein Spiel aktiv laufen.
- Die Spielerliste muss zwischen 2 und 4 Spieler enthalten.
- Die Rundenanzahl muss zwischen 2 und 7 liegen.

Nachbedingungen:

- Ein vollständiges Kartendeck wurde erstellt und gemischt.
- Jeder Spieler besitzt 2 verdeckte und 3 offene Karten.
- 3 Karten liegen offen in der Mitte.
- Nachziehstapel wurde erstellt, der Ablagestapel ist leer.
- Ein Startspieler wurde bestimmt.
- Das Spiel befindet sich im laufenden Zustand.

Parameter:

- players: Liste der teilnehmenden Spieler.
- amountRounds: Anzahl der zu spielenden Runden.

Gültige Wertebereiche für Parameter:

- players: Größe zwischen 2 und 4.
- amountRounds: Ganzzahl zwischen 2 und 7.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- IllegalStateException: Wird geworfen, wenn bereits ein Spiel läuft.
- IllegalArgumentException: Wird geworfen, wenn die Spieleranzahl oder Rundenzahl außerhalb des gültigen Bereichs liegt.

endGame() : Unit

Allgemeine Beschreibung:

Die Methode endGame beendet das aktuelle Spiel nach Abschluss der festgelegten Rundenzahl und berechnet die Rangfolge der Spieler anhand ihrer Handkarten.

Vorbedingungen:

- Das Spiel muss laufen.
- Die maximale Rundenanzahl muss erreicht sein.

Nachbedingungen:

- Alle Hände der Spieler wurden ausgewertet.
- Eine Rangliste wurde erstellt.
- Das Spiel befindet sich im beendeten Zustand.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn kein Spiel läuft oder die Rundenzahl noch nicht erreicht wurde.

updateLog() : Unit

Allgemeine Beschreibung:

Die Methode updateLog ergänzt den Spiel-Log um Informationen zur zuletzt ausgeführten Aktion, sodass alle Spielzüge nachvollziehbar sind.

Vorbedingungen:

- Ein Spiel muss laufen.
- Eine Aktion muss zuvor stattgefunden haben.

Nachbedingungen:

- Der Log wurde um einen neuen Eintrag erweitert.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn kein Spiel aktiv ist.

createDrawStack() : Stack<Card>

Allgemeine Beschreibung:

Die Methode createDrawStack erstellt ein vollständiges Kartendeck mit 52 Karten, mischt dieses und gibt es als Nachziehstapel zurück.

Vorbedingungen:

Keine.

Nachbedingungen:

- Ein gemischter Stapel mit 52 Karten wurde erzeugt.

Parameter:

Keine.

Ergebnis:

Gibt einen gemischten `Stack<Card>` zurück.

Fehlerfälle:

Keine.

refillDrawStack(discardStack: Stack<Card>) : Stack<Card>

Allgemeine Beschreibung:

Die Methode refillDrawStack mischt den Ablagestapel und verwendet ihn als neuen Nachziehstapel, wenn der ursprüngliche Nachziehstapel leer ist.

Vorbedingungen:

- Der Nachziehstapel ist leer.
- Der Ablagestapel enthält mindestens eine Karte.

Nachbedingungen:

- Der Ablagestapel wurde gemischt.
- Ein neuer Nachziehstapel wurde erstellt.
- Der Ablagestapel ist danach leer.

Parameter:

- discardStack: aktueller Ablagestapel.

Gültige Wertebereiche für Parameter:

- discardStack muss mindestens eine Karte enthalten.

Ergebnis:

Gibt einen neuen Stack<Card> zurück.

Fehlerfälle:

- IllegalStateException: Wird geworfen, wenn der Ablagestapel leer ist.

drawCard() : Unit

Allgemeine Beschreibung:

Die Methode drawCard zieht die oberste Karte vom Nachziehstapel. Falls dieser leer ist, wird automatisch refillDrawStack aufgerufen.

Vorbedingungen:

- Das Spiel muss laufen.
- Es existiert ein Nachziehstapel oder ein nicht leerer Ablagestapel.

Nachbedingungen:

- Eine Karte wurde vom Nachziehstapel gezogen.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn weder Nachzieh- noch Ablagestapel Karten enthalten.

evaluateCards() : Int

Allgemeine Beschreibung:

Die Methode evaluateCards bewertet die Hand eines Spielers gemäß den Pokerregeln und gibt deren Gesamtwert zurück.

Vorbedingungen:

- Das Spiel muss gestartet sein.
- Die Hand muss genau 5 Karten enthalten.

Nachbedingungen:

- Die Hand wurde analysiert und einer Pokerkombination zugeordnet.

Parameter:

Keine.

Ergebnis:

Gibt einen Integer zurück, der den Kartenwert der Hand darstellt.

Fehlerfälle:

- `IllegalArgumentException`: Wird geworfen, wenn die Hand nicht genau 5 Karten enthält.

addPlayer(name: String) : Unit

Allgemeine Beschreibung:

Die Methode addPlayer erstellt einen neuen Spieler mit dem angegebenen Namen und fügt ihn der Spielerliste hinzu.

Vorbedingungen:

- Das Spiel darf noch nicht gestartet sein.
- Der Name darf nicht leer sein.

Nachbedingungen:

- Ein neuer Spieler wurde der Spielerliste hinzugefügt.

Parameter:

- name: Name des Spielers.

Gültige Wertebereiche für Parameter:

- name darf kein leerer String sein.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- IllegalStateException: Wird geworfen, wenn das Spiel bereits gestartet wurde.
- IllegalArgumentException: Wird geworfen, wenn der Name ungültig ist.

chooseRounds(amountRounds: Int) : Unit

Allgemeine Beschreibung:

Die Methode chooseRounds legt die Anzahl der zu spielenden Runden fest.

Vorbedingungen:

- Das Spiel darf noch nicht gestartet sein.

Nachbedingungen:

- Die Rundenzahl wurde gespeichert.

Parameter:

- amountRounds: Anzahl der Runden.

Gültige Wertebereiche für Parameter:

- amountRounds muss zwischen 2 und 7 liegen.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- IllegalStateException: Wird geworfen, wenn das Spiel bereits läuft.
- IllegalArgumentException: Wird geworfen, wenn amountRounds außerhalb des gültigen Bereichs liegt.

endTurn() : Unit

Allgemeine Beschreibung:

Die Methode endTurn beendet den aktuellen Spielzug eines Spielers, nachdem beide Aktionen ausgeführt wurden oder keine weiteren Aktionen mehr möglich sind. Anschließend wird der nächste Spieler bestimmt. Falls alle Spieler einer Runde ihren Zug abgeschlossen haben, wird die Runde erhöht und geprüft, ob das Spiel beendet werden muss.

Vorbedingungen:

- Das Spiel muss laufen
- Der aktuelle Spieler befindet sich in einem aktiven Zug.
- Es dürfen keine Aktionen mehr offen sein oder der Zug wird regelkonform beendet.

Nachbedingungen:

- Der Zug des aktuellen Spielers wurde abgeschlossen.
- Der nächste Spieler ist an der Reihe.
- Falls notwendig, wurde die Rundenanzahl erhöht.
- Wenn die maximale Rundenanzahl erreicht wurde, wird endGame ausgelöst.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn kein Spiel aktiv ist oder der Zug nicht beendet werden darf.

PlayerActionService

skipTurn() : Unit

Allgemeine Beschreibung:

Die Methode skipTurn überspringt die verbleibenden Aktionen des aktuellen Spielers.

Vorbedingungen:

- Das Spiel muss laufen.
- Der Spieler ist an der Reihe.
- Mindestens eine Aktion ist noch verfügbar.

Nachbedingungen:

- Die verbleibenden Aktionen wurden auf 0 gesetzt.
- Der Log wurde aktualisiert.
- Falls keine Aktionen mehr übrig sind, wird zum nächsten Spieler gewechselt.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn das Spiel nicht läuft oder der Spieler nicht an der Reihe ist.

push(direction: Int) : Unit

Allgemeine Beschreibung:

Die Methode push verschiebt die drei offenen Karten auf dem Tisch in die angegebene Richtung. Anschließend wird eine neue Karte vom Nachziehstapel gezogen und eingefügt. Die äußere Karte wird auf den Ablagestapel gelegt.

Vorbedingungen:

- Das Spiel muss laufen.
- Der Spieler ist an der Reihe.
- Mindestens eine Aktion ist verfügbar.
- direction ist gültig.

Nachbedingungen:

- Die Karten wurden in die gewählte Richtung verschoben.
- Eine neue Karte wurde gezogen.
- Die äußere Karte liegt auf dem Ablagestapel.
- Eine Aktion wurde reduziert.
- Der Log wurde aktualisiert.

Parameter:

- direction: Gibt die Richtung an, in die geschoben wird.

Gültige Wertebereiche für Parameter:

- direction: 0 = links, 1 = rechts.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- IllegalStateException: Wird geworfen, wenn das Spiel nicht läuft oder der Spieler nicht an der Reihe ist.
- IllegalArgumentException: Wird geworfen, wenn direction ungültig ist.

switch(cardsToSwitch: List<Card>) : Unit

Allgemeine Beschreibung:

Die Methode switch tauscht ausgewählte Karten des Spielers mit Karten auf dem Tisch. Es können entweder eine oder drei Karten getauscht werden.

Vorbedingungen:

- Das Spiel muss laufen.
- Der Spieler ist an der Reihe.
- Mindestens eine Aktion ist verfügbar.
- cardsToSwitch enthält gültige Karten.

Nachbedingungen:

- Die ausgewählten Karten wurden getauscht.
- Eine Aktion wurde reduziert.
- Der Log wurde aktualisiert.

Parameter:

- cardsToSwitch: Liste der zum Tausch ausgewählten Karten.

Gültige Wertebereiche für Parameter:

- cardsToSwitch enthält entweder 1 oder 3 Karten.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn das Spiel nicht läuft oder der Spieler nicht an der Reihe ist.
- `IllegalArgumentException`: Wird geworfen, wenn keine oder ungültige Karten übergeben wurden.

reduceAction() : Unit

Allgemeine Beschreibung:

Die Methode reduceAction reduziert die verbleibenden Aktionen eines Spielers um eins.

Vorbedingungen:

- Das Spiel muss laufen.
- Der Spieler besitzt mindestens eine verbleibende Aktion.

Nachbedingungen:

- Die Anzahl der verbleibenden Aktionen wurde um eins reduziert.

Parameter:

Keine.

Ergebnis:

Diese Methode hat keinen Rückgabewert (Unit).

Fehlerfälle:

- `IllegalStateException`: Wird geworfen, wenn keine Aktionen mehr verfügbar sind.